

MEG/EEG Preprocessing Pipeline Documentation

Version 1.4 (2025-11-05)

Author: Gregory McCarthy with AI assistance

Checkpoint Format: 1.0

Table of Contents

- 1. Introduction
 - 2. Core Design Principles
 - 3. Stage 1: Automated Heavy Processing
 - 4. Stage 2: Interactive Finalization
 - 5. Checkpoint System
 - 6. Hybrid Processing Mode
 - 7. Filtering Philosophy
 - 8. File-Naming and Normalization
 - 9. Directory Layout
 - 10. Environment Setup
 - 11. Execution Modes
 - 12. Quality-Control Framework
 - 13. Troubleshooting Guide
 - 14. Performance Optimization
 - 15. Validation and Recovery
 - 16. Customization
 - 17. Complete Production Configuration
 - 18. Version History
 - 19. References and Resources
-

1 Introduction

This pipeline provides a reproducible, BIDS-compliant framework for preprocessing MEG and EEG data using **MNE-Python**.

It divides processing into two clearly defined stages:

- **Stage 1 – Automated Heavy Processing:** performs deterministic, resource-intensive computations without user interaction.
- **Stage 2 – Interactive Finalization:** applies expert judgment for artifact review, ICA selection, and final filtering.

Both stages are linked by a robust **checkpoint system** that prevents redundant computation and guarantees reproducibility.

Primary goals

- Standardize complex preprocessing across datasets and laboratories.

- Preserve signal integrity while documenting every operation.
- Support high-performance (HPC) and hybrid local/HPC workflows.
- Enable full audit trails for scientific transparency.

2 Core Design Principles

1. **Two-Stage Architecture** – separates automation from expert review.
2. **Checkpoint Reproducibility** – all parameters stored in YAML/JSON manifests.
3. **Hybrid Operation** – data fetched and pushed between HPC and local workstations.
4. **BIDS Compliance** – tolerant of imperfect inputs, strict on standardized outputs.
5. **Transparent Filtering** – all filtering explicitly logged; raw data preserved.
6. **Conservative Bias** – no irreversible decisions before human review.
7. **Scalability** – from single-subject testing to hundreds of sessions.

3 Stage 1 – Automated Heavy Processing

Stage 1 executes all deterministic steps that can safely run unattended. It is optimized for HPC batch jobs or overnight local runs.

3.1 Major Operations

- Head-position estimation from continuous **HPI** signals.
- Maxwell filtering (**SSS/tSSS**) for environmental noise suppression.
- Line-noise notch filtering.
- Automated artifact detection using **AutoReject** (detection-only mode).
- Computation of objective QC metrics (pre- and post-Maxwell).
- Generation of diagnostic figures and a reproducible checkpoint.

3.2 Execution Context

Stage 1 may be run:

- on HPC (SLURM) in headless mode; or
- locally for development testing.

All plotting uses the **Agg** backend to ensure compatibility with non-display environments.

3.3 Plotting Behavior

Stage	Environment	Matplotlib Backend	Output
1	HPC or batch	Agg	Headless; writes PNGs only
2	Interactive	Qt6Agg (macOS) / QtAgg (Linux)	GUI + PNG export

Stage 1 uses **Agg** without committing to a GUI backend. All figures are stored as static PNGs in:

derivatives/preprocessing/sub-*/ses-*/meg/plots/

3.4 Raw Data Loading

- Automatically detects split MEGIN files (`_meg.fif`, `_meg-1.fif`, `_split-01_meg.fif` ...).
- Concatenates and memory-maps data for efficient I/O.
- Validates BIDS entities and metadata integrity.
- Extracts subject, session, task, and run information from configuration.

```
subject: "001"  
session: "01"  
run: "02"  
task: "fairy"  
bids_root: /path/to/BIDS
```

3.5 Head-Position Estimation and Movement Compensation

Problem: Subject motion changes sensor–source geometry, degrading localization.

Solution: Continuous HPI monitoring and physics-based compensation.

- Extracts coil positions frame-by-frame.
- Computes translation and rotation traces.
- Derives movement velocity histograms and fit quality metrics.
- Applies motion-compensated **tSSS** reconstruction.

 Figure 1. Head-movement diagnostics

Quality metrics produced

- RMS displacement
- Coil fit residuals
- Shielding factors
- Movement velocity summaries

3.6 Maxwell Filtering (SSS/tSSS)

Theoretical basis: magnetic field decomposition into internal (brain) and external (environment) components via spherical harmonics.

Implementation:

- Performs SSS reconstruction using system calibration (`cal/ct_sparse`).
- Applies tSSS to suppress temporally correlated external noise.
- Interpolates bad channels via field reconstruction.

Configuration example

```
calibration_file: cal/sss_cal_factory_20230619.dat  
cross_talk_file: cal/ct_sparse_triux2.fif  
head_movement:
```

```
enabled: true
head_position_origin: null
```

3.7 EEG Channel Standardization

EEG within MEG systems often lacks consistent labels.

Pipeline steps:

1. Apply standard montage.
2. Validate electrode positions.
3. Convert to consistent naming conventions.
4. Detect and flag bad electrodes.
5. Verify reference configuration.

 Figure 2. EEG montage validation

3.8 Metadata Repair System

Real-world datasets frequently include inconsistent or missing metadata.

A flexible repair subsystem allows both automated fixes and dataset-specific patches.

Repair categories

- Systematic: channel-type correction, coordinate alignment.
- Expert: user-defined Python code.

```
metadata_fixes:
  fix_non_eeg_channels: {}
  fix_generic: {}
  expert_patch: |
    # Custom patch example
    print("✅ Expert patch executed")
```

3.9 Line-Noise Suppression

Strategy

- Identify 50/60 Hz peaks and harmonics.
- Apply adaptive-width notch filters.
- Verify via pre/post spectral plots.

 Figure 3. Line-noise removal example

3.10 AutoReject: Automated Artifact Detection

Reference: Jas et al. (2017).

Objective, cross-validated thresholding replaces subjective manual inspection.

Three-layer detection

- 1. RANSAC bad-channel detection.
- 2. Sensor-specific AutoReject thresholds.
- 3. Global epoch consensus voting.

Detection is annotation-only; no interpolation or rejection occurs automatically.

```
autoreject:
  enabled: true
  which_types: [eeg, mag]
  filter:
    highpass: 1.0
    lowpass: 40.0
  epoch:
    duration: 2.0
  cv_folds: 5
  thresh_method: bayesian_optimization
  n_interpolate: [0]
  consensus_thresh: 0.3
  global_epoch_thresh: 0.5
```

Figure 4. AutoReject diagnostics

3.11 Quality-Control Metrics

QC metrics are computed both **pre-** and **post-Maxwell**.

Category	Example Metrics
Amplitude	RMS, 95 % peak-to-peak
Spectral	1/f slope, alpha-band power, line-noise ratios
Statistical	Kurtosis, variance ratios
Spatial	Shielding factor, movement distance
Improvement	pre/post amplitude and variance ratios

All metrics are written to the manifest and summarized in QC figures.

3.12 Checkpoint Creation

At Stage 1 completion:

- **FIF:** *_desc-parproc_meg.fif – Maxwell + notch, full bandwidth.

- **YAML Manifest:** parameters, hashes, QC results, artifact lists.
 - **PNG Diagnostics:** PSD, movement, AutoReject, etc.
-

4 Stage 2 – Interactive Finalization

Stage 2 begins by loading and validating the checkpoint, then enables expert review.

4.1 Checkpoint Validation

- Confirm presence of both FIF and YAML.
- Verify version compatibility and integrity.
- Detect configuration drift (hash mismatch).

4.2 Interactive Review

Two complementary interfaces:

Channel Review

- Time-series view with zoom/pan.
- Topographic sensor map with bad-channel markers.
- Butterfly and individual traces.
- Spectral plots per sensor type.

Epoch Review

- Overlay of epochs with artifact flags.
- Toggle to reject/restore epochs.

 Figure 5. Interactive channel review

All edits are timestamped and logged to YAML/JSON.

4.3 Independent Component Analysis (ICA)

Goal: separate neural sources from ocular, cardiac, and muscular artifacts.

Reference: Hyvärinen & Oja (2000).

Procedure

1. Create 1–40 Hz filtered copy and optionally resample.
2. Fit ICA separately for EEG and MEG.
3. Apply weights to full-band original data (preserves all frequencies).
4. Review and exclude artifact components.

 Figure 6. ICA component spectra

```
ica_preprocessing:
  eeg:
    highpass: 1.0
```

```
lowpass: 40.0
resample_hz: 200
max_ica_duration_sec: 1800
random_state: 97
meg:
  highpass: 1.0
  lowpass: 40.0
  resample_hz: 200
  max_ica_duration_sec: 1800
  random_state: 97
```

Automated aids (correlation with EOG/ECG channels, spectral fingerprints) assist classification.

4.4 Event Detection and Validation

Supports flexible trigger encoding.

- Bit-mask extraction (`stim_include_mask`).
- Merge of digital and annotation events.
- Sequence and count validation.

```
stim_include_mask: 0xFFFF
```

4.5 Final Filtering and Channel Management

- Optional high/low/band-pass filters per analysis.
- Drop auxiliary channels (`hpi`, `hlc`, `coil`).
- Enforce consistent channel order.

```
final_filter:
  highpass: null
  lowpass: null
  resample_hz: null
  drop_channel_types: [hpi, hlc, coil]
```

4.6 Final Outputs

Artifact	Description
<code>_desc-preproc_meg.fif</code>	Fully cleaned, ready for analysis
<code>_desc-preprocICA{eeg,meg}.fif</code>	ICA decomposition matrices
<code>_log.yaml</code> , <code>_log.json</code>	Complete logs

Artifact	Description
plots/	Final diagnostic figures

5 Checkpoint System – Architecture and Purpose

Each checkpoint = **data + manifest** pair ensuring reproducibility and resumability.

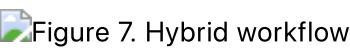
Contents

- Processing parameters and configuration hash.
- Lists of artifacts, bad channels, and QC metrics.
- References to calibration files and diagnostic figures.

Advantages

- Re-use without recomputing heavy operations.
- Enables collaborative review.
- Guarantees provenance for publication.

6 Hybrid Processing Mode



Motivation: combine HPC power with local interactivity.

Workflow

1. **Prefetch:** sync raw and prior derivatives from HPC → local.
2. **Local processing:** run both stages interactively.
3. **Pushback:** sync completed derivatives back to HPC.

```
remote_io:
  enabled: true
  hpc_host: transfer.cluster.edu
  hpc_user: username
  remote_bids_root: /gpfs/project/bids
  local_temp_dir: ./temp
```

Benefits: minimal network latency, authoritative central storage, and team collaboration.

7 Filtering Philosophy

No redundant filtering; algorithm-specific temporary copies only.
Original broadband data preserved for final outputs.

Step	Filter	Data Preserved	Rationale
------	--------	----------------	-----------

Step	Filter	Data Preserved	Rationale
AutoReject	1–40 Hz	No	Stabilize thresholds
ICA fit	1–40 Hz + resample	No	Reduce noise & load
ICA apply	None	Yes	Preserve bandwidth
Final output	User-defined	Yes	Analysis-specific

8 File-Naming and Normalization

Pipeline accepts irregular input names (e.g., 'sub-1') but **normalizes outputs** to canonical BIDS:

```
sub-001_ses-01_task-faces_run-01_desc-parproc_meg.fif
sub-001_ses-01_task-faces_run-01_desc-preproc_meg.fif
sub-001_ses-01_task-faces_run-01_desc-preprocICAmeg.fif
```

This behavior guarantees cross-platform reproducibility and integration with downstream BIDS apps.

9 Directory Layout

Input (BIDS)

```
bids_root/
├── dataset_description.json
├── participants.tsv
├── sub-001/
│   ├── ses-01/
│   │   └── meg/
│   │       ├── sub-001_ses-01_task-faces_run-01_meg.fif
│   │       ├── sub-001_ses-01_task-faces_run-01_meg.json
│   │       └── sub-001_ses-01_task-faces_run-01_channels.tsv
```

Output (Derivatives)

```
derivatives/preprocessing/
├── sub-001/ses-01/meg/
│   ├── *_desc-parproc_meg.fif
│   ├── *_desc-parproc_meg_manifest.yaml
│   ├── *_desc-preproc_meg.fif
│   ├── *_desc-preprocICA{eeg,meg}.fif
│   ├── *_log.yaml / *_log.json
│   └── plots/*.png
```

10 Environment Setup

10.1 Conda Environment

A preconfigured file exists at `env/mne_pipeline_env.yaml`.

```
conda env create -f env/mne_pipeline_env.yaml
conda activate mne-pipeline
```

10.2 Manual Installation

```
pip install "mne>=1.6.0" "mne-bids>=0.14" "autoreject>=0.4.3"
pip install "pyyaml>=6.0" "numpy>=1.24" "scipy>=1.10" "matplotlib>=3.7"
pip install "pandas>=2.0" "joblib>=1.3" "scikit-learn>=1.3"
```

10.3 Environment Variables

Variable	Typical Value	Purpose
MPLBACKEND	Agg (Stage 1) / QtAgg (Stage 2)	Backend control
QT_QPA_PLATFORM	xcb	Qt display under Linux
OMP_NUM_THREADS	8	Thread control
MKL_NUM_THREADS	8	Thread control
OPENBLAS_NUM_THREADS	8	Thread control
NUMEXPR_NUM_THREADS	8	Thread control
MNE_USE_CUDA	false	Disable GPU unless configured
MNE_LOGGING_LEVEL	INFO	Verbose logging

11 Execution Modes

Mode	Use Case	Command
Local	Full interactive processing	<code>python preprocess_meg.py config/sub-001.yaml</code>
Stage 1 only	HPC batch job	<code>python preprocess_meg.py config/sub-001.yaml --force-stage stage1</code>
Stage 2 resume	Interactive from checkpoint	<code>python preprocess_meg.py config/sub-001.yaml --force-stage stage2</code>

Mode	Use Case	Command
SLURM batch	Array processing	<code>sbatch scripts/meg_stage1.slurm</code>
Hybrid	Prefetch→process→pushback	YAML <code>remote_io</code> enabled

Example SLURM script:

```
#!/bin/bash
#SBATCH --job-name=meg_stage1
#SBATCH --time=04:00:00
#SBATCH --mem=64G
#SBATCH --cpus-per-task=8
export MPLBACKEND=Agg
python preprocess_meg.py config/slurm.yaml --force-stage stage1
```

12 Quality-Control Framework

QC operates at multiple levels:

12.1 Pre-Processing Baseline

- Raw amplitude distributions
- Channel integrity
- Recording metadata validation

12.2 Post-Maxwell Improvement

- Noise suppression ratios
- Spectral cleaning metrics
- Shielding factor gains

12.3 Post-Artifact Removal

- Artifact reduction statistics
- Signal-to-noise improvements
- Component removal impacts

Figure 8. QC report summary

Quality Grades

Grade	Description
A	Excellent – minimal artifacts
B	Good – acceptable SNR
C	Fair – marginal SNR

Grade	Description
D	Poor – low SNR
F	Fail – unusable data

13 Troubleshooting Guide

13.1 Common Issues and Fixes

Problem	Symptom	Diagnosis	Solution
Checkpoint not found	Stage 2 attempts to run but can't find checkpoint	Missing or misnamed <code>_desc-parproc_meg.fif</code>	Verify derivatives path, subject/session/run, and file permissions
Memory error	"Killed" or "Out of Memory" during AutoReject or ICA	Data too long or too many channels	Downsample, shorten epochs, or set <code>max_ica_duration_sec</code>
No display	GUI windows never appear	<code>\$DISPLAY</code> undefined; backend Agg forced	Export <code>\$DISPLAY</code> , ensure VNC/X11, or run Stage 1 only on cluster
Calibration failure	Maxwell filter aborts	Wrong calibration/crosstalk path	Use absolute paths; confirm read access
Network hang in hybrid mode	Prefetch/pushback stalls	Broken SSH/rsync	Test with small transfer; check SSH keys and timeouts
AutoReject anomaly	All channels marked bad or none detected	Inappropriate filtering or too few epochs	Adjust filter, extend epoch duration, or increase <code>cv_folds</code>

Example diagnostics

```
# Check backend
python -c "import matplotlib; print(matplotlib.get_backend())"
# Verify calibration
ls cals/
# Test rsync
rsync -av testfile user@hpc:/tmp/
```

14 Performance Optimization

14.1 CPU and Memory

```
export OMP_NUM_THREADS=8
export MKL_NUM_THREADS=8
export OPENBLAS_NUM_THREADS=8
```

- Use local SSD scratch for temporary files.
- Prefer memory-mapping over full loads (`preload=False`).
- For large studies, process subjects in batches.

14.2 Speed Tips

Goal	Setting
Faster AutoReject	Use <code>random_search</code> and fewer folds
Faster ICA	Limit <code>max_ica_duration_sec</code> and resample to 100–200 Hz
Quick preview	Run Stage 1 only with reduced epoch length
Faster I/O	Avoid network mounts; cache locally

15 Validation and Recovery

15.1 Data Integrity Checks

```
import mne, yaml
raw = mne.io.read_raw_fif("checkpoint.fif")
with open("manifest.yaml") as f:
    man = yaml.safe_load(f)
assert len(raw.info["bads"]) == len(man["artifacts"]["bad_channels"])
print("✓ checkpoint valid")
```

15.2 Pipeline State Recovery

1. Find partial outputs created recently:


```
find derivatives/ -name "*.fif" -mmin -60
```
2. Test checkpoint integrity.
3. Resume Stage 2 if valid, else re-run Stage 1.
4. Use backup checkpoint if available (`backup_before_overwrite: true`).

16 Customization

The pipeline exposes hooks for laboratory-specific processing.

16.1 Custom Artifact Detection

```
# in expert_patch section
def custom_artifact_detection(raw):
    # user logic
    return bad_channels, bad_epochs
raw.info["bads"].extend(custom_artifact_detection(raw)[0])
```

16.2 Additional Filtering

```
metadata_fixes:
  expert_patch: |
    from mne.filter import filter_data
    raw._data = filter_data(raw._data, raw.info['sfreq'], l_freq=0.3,
h_freq=None)
```

17 Complete Production Configuration Example

```
# =====
# MEG/EEG Preprocessing Pipeline – Production Configuration
# Version 1.4
# =====

subject: "001"
session: "01"
run: "02"
task: "experiment"
bids_root: /data/project/bids

remote_io:
  enabled: false
  hpc_host: transfer.cluster.edu
  hpc_user: username
  remote_bids_root: /hpc/project/bids
  local_temp_dir: ./temp

head_movement:
  enabled: true

calibration_file: cals/sss_cal.dat
cross_talk_file: cals/ct_sparse.fif
head_position_origin: null
line_freq: 60.0

eeg_handling:
  montage: montage/standard_64ch.sfp

interactive_bad_channels: true
interactive_ica: true
```

```
manual_bad_channels: []
ica_exclude_eeg: []
ica_exclude_meg: []

autoreject:
  enabled: true
  which_types: [eeg, mag, grad]
  filter:
    highpass: 1.0
    lowpass: 40.0
  epoch:
    duration: 2.0
  cv_folds: 5
  thresh_method: bayesian_optimization
  n_interpolate: [0]
  consensus_thresh: 0.3
  global_epoch_thresh: 0.5

ica_preprocessing:
  eeg:
    highpass: 1.0
    lowpass: 40.0
    resample_hz: 200
    max_ica_duration_sec: 1800
    random_state: 97
  meg:
    highpass: 1.0
    lowpass: 40.0
    resample_hz: 200
    max_ica_duration_sec: 1800
    random_state: 97

stim_include_mask: 0xFFFF

final_filter:
  highpass: null
  lowpass: null
  resample_hz: null
  drop_channel_types: [hpi, hlc, coil]

metadata_fixes:
  fix_non_eeg_channels: {}
  fix_generic: {}
  expert_patch: |
    print("✅ Metadata repair completed")

checkpoint:
  enabled: true
  exit_after_checkpoint: false
  derivatives_root: derivatives
  pipeline_name: preprocessing
  atomic_writes: true
  allow_resume_with_different_yaml: true
  validate_checkpoint_integrity: true
```

```
backup_before_overwrite: false
max_checkpoint_age_hours: 168

repro:
  random_state_global: 42
  record_environment: true
```

18 Version History

Version	Date	Changes
1.4	2025-11-05	Added explicit backend control (Agg vs Qt6Agg/QtAgg); expanded environment setup; clarified BIDS normalization and fault-tolerance; merged environment / execution guidance.
1.3	2025-09-02	Enhanced AutoReject and QC documentation.
1.2	2025-08-15	Introduced hybrid mode and Stage separation.
1.1	2025-07-01	Added checkpoint architecture.
1.0	2025-06-01	Initial release.

19 References and Resources

Core Tools

- [MNE-Python](#)
- [MNE-BIDS](#)
- [AutoReject](#)

Key Papers

- Jas et al. (2017). *Autoreject: Automated artifact rejection for MEG and EEG data.*
- Taulu & Simola (2006). *Spatiotemporal signal space separation method.*
- Hyvärinen & Oja (2000). *Independent component analysis: algorithms and applications.*

Community Resources

- [MNE Forum](#)
- [FieldTrip Toolbox](#)
- [EEGLAB](#)

